

LAB 10 : CE159 - Prashant

Aim : Creating Microservices

Create below Microservices in same solution :

1. **StudentAdmission** : It maintains StudentID, Name, AdmissionDate and AdmissionBranch details.
2. **StudentAttendance** : It maintains StudentID, Semester and Percentage_Attendance details.

- Create basic CRUD controllers
- Use Repository pattern while developing above services.

StudentAdmissionService/

Models/StudentAdmission.cs:

```
namespace StudentAdmissionService.Models
{
    public class StudentAdmission
    {
        public int StudentID { get; set; }
        public string Name { get; set; }
        public DateTime AdmissionDate { get; set; }
        public string AdmissionBranch { get; set; }
    }
}
```

Repository/

IStudentAdmissionRepository.cs:

```
using StudentAdmissionService.Models;
namespace StudentAdmissionService.Repository
{
    public interface IStudentAdmissionRepository
    {
        IEnumerable<StudentAdmission> GetAll();
        StudentAdmission GetByld(int id);
        StudentAdmission Add(StudentAdmission student);
        StudentAdmission Update(int id, StudentAdmission student);
        bool Delete(int id);
    }
}
```

StudentAdmissionRepository.cs:

```
using StudentAdmissionService.Models;

namespace StudentAdmissionService.Repository
{
    public class StudentAdmissionRepository : IStudentAdmissionRepository
    {

```

```

private static List<StudentAdmission> _students = new List<StudentAdmission>();

public IEnumerable<StudentAdmission> GetAll()
{
    return _students;
}

public StudentAdmission GetById(int id)
{
    return _students.FirstOrDefault(s => s.StudentID == id);
}

public StudentAdmission Add(StudentAdmission student)
{
    _students.Add(student);
    return student;
}

public StudentAdmission Update(int id, StudentAdmission student)
{
    var existing = _students.FirstOrDefault(s => s.StudentID == id);
    if (existing == null) return null;

    existing.Name = student.Name;
    existing.AdmissionDate = student.AdmissionDate;
    existing.AdmissionBranch = student.AdmissionBranch;

    return existing;
}

public bool Delete(int id)
{
    var student = _students.FirstOrDefault(s => s.StudentID == id);
    if (student == null) return false;

    _students.Remove(student);
    return true;
}
}
}

```

Program.cs:

```

using StudentAdmissionService.Repository;
namespace StudentAdmissionService
{
    public class Program
    {
        public static void Main(string[] args)

```

```

{
    var builder = WebApplication.CreateBuilder(args);

    builder.Services.AddControllers();
    builder.Services.AddEndpointsApiExplorer();
    builder.Services.AddSwaggerGen();

    builder.Services.AddSingleton<IStudentAdmissionRepository, StudentAdmissionRepository>();
    var app = builder.Build();

    if (app.Environment.IsDevelopment())
    {
        app.UseSwagger();
        app.UseSwaggerUI();
    }
    app.UseHttpsRedirection();
    app.UseAuthorization();
    app.MapControllers();
    app.Run();
}
}
}

```

Controllers/StudentAdmissionController.cs:

```

using Microsoft.AspNetCore.Mvc;
using StudentAdmissionService.Models;
using StudentAdmissionService.Repository;

namespace StudentAdmissionService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentAdmissionController : ControllerBase
    {
        private readonly IStudentAdmissionRepository _repository;

        public StudentAdmissionController(IStudentAdmissionRepository repository)
        {
            _repository = repository;
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(_repository.GetAll());
        }

        [HttpGet("{id}")]
        public IActionResult GetById(int id)

```

```

{
    var student = _repository.GetById(id);
    if (student == null) return NotFound();
    return Ok(student);
}

[HttpPost]
public IActionResult Add(StudentAdmission student)
{
    return Ok(_repository.Add(student));
}

[HttpPut("{id}")]
public IActionResult Update(int id, StudentAdmission student)
{
    var updated = _repository.Update(id, student);
    if (updated == null) return NotFound();
    return Ok(updated);
}

[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    if (!_repository.Delete(id))
        return NotFound();

    return Ok("Deleted Successfully");
}
}
}

```

POST	/api/StudentAdmission
Parameters	
No parameters	
Request body	
<pre> { "studentID": 1, "name": "Prashant", "admissionDate": "2026-03-02T04:12:58.523Z", "admissionBranch": "CE" } </pre>	

Code	Details
200	Response body <pre>{ "studentID": 1, "name": "Prashant", "admissionDate": "2026-03-02T04:12:58.523Z", "admissionBranch": "CE"}</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 02 Mar 2026 04:13:26 GMT server: Kestrel</pre>

GET /api/StudentAdmission

Parameters

No parameters

Execute

Code	Details
200	Response body <pre>[{ "studentID": 1, "name": "Prashant", "admissionDate": "2026-03-02T04:12:58.523Z", "admissionBranch": "CE" }, { "studentID": 2, "name": "Prem", "admissionDate": "2023-08-02T00:00:00", "admissionBranch": "IT" }]</pre> Response headers

PUT

/api/StudentAdmission/{id}

Parameters

Name	Description
------	-------------

id * required

integer(\$int32)

(path)

Request body

```
{
  "studentID": 1,
  "name": "Prashant",
  "admissionDate": "2026-03-02T04:16:52.403Z",
  "admissionBranch": "CH"
}
```

Code

Details

200

Response body

```
{
  "studentID": 1,
  "name": "Prashant",
  "admissionDate": "2026-03-02T04:16:52.403Z",
  "admissionBranch": "CH"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 02 Mar 2026 04:17:14 GMT
server: Kestrel
```

GET

/api/StudentAdmission/{id}

Parameters

Name	Description
------	-------------

id * required

integer(\$int32)

(path)

Execute

200

Response body

```
{
  "studentID": 1,
  "name": "Prashant",
  "admissionDate": "2026-03-02T04:16:52.403Z",
  "admissionBranch": "CH"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 02 Mar 2026 04:18:21 GMT
server: Kestrel
```

DELETE /api/StudentAdmission/{id}

Parameters

Name	Description
------	-------------

id * required integer(\$int32) (path)	2
--	--------------------------------

Execute

200

Response body

```
Deleted Successfully
```

Response headers

```
content-type: text/plain; charset=utf-8
date: Mon, 02 Mar 2026 04:19:01 GMT
server: Kestrel
```

StudentAttendanceService/

Models/StudentAttendance.cs:

```
namespace StudentAttendanceService.Models
{
    public class StudentAttendance
    {
        public int StudentID { get; set; }
        public int Semester { get; set; }
        public double Percentage_Attendance { get; set; }
    }
}
```

Repository/

IStudentAttendanceRepository.cs:

```
using StudentAttendanceService.Models;
namespace StudentAttendanceService.Repository
{
    public interface IStudentAttendanceRepository
    {
        IEnumerable<StudentAttendance> GetAll();
        StudentAttendance GetById(int id);
        StudentAttendance Add(StudentAttendance attendance);
        StudentAttendance Update(int id, StudentAttendance attendance);
        bool Delete(int id);
    }
}
```

StudentAttendanceRepository.cs:

```
using StudentAttendanceService.Models;
namespace StudentAttendanceService.Repository
{
    public class StudentAttendanceRepository : IStudentAttendanceRepository
    {
        private static List<StudentAttendance> _attendance = new List<StudentAttendance>();

        public IEnumerable<StudentAttendance> GetAll()
        {
            return _attendance;
        }

        public StudentAttendance GetById(int id)
        {
            return _attendance.FirstOrDefault(a => a.StudentID == id);
        }

        public StudentAttendance Add(StudentAttendance attendance)
        {
            _attendance.Add(attendance);
            return attendance;
        }

        public StudentAttendance Update(int id, StudentAttendance attendance)
        {
            var existing = _attendance.FirstOrDefault(a => a.StudentID == id);
            if (existing == null) return null;

            existing.Semester = attendance.Semester;
            existing.Percentage_Attendance = attendance.Percentage_Attendance;

            return existing;
        }
    }
}
```

```

    }

    public bool Delete(int id)
    {
        var existing = _attendance.FirstOrDefault(a => a.StudentID == id);
        if (existing == null) return false;

        _attendance.Remove(existing);
        return true;
    }
}
}

```

Controllers/StudentAttendanceController.cs:

```

using Microsoft.AspNetCore.Mvc;
using StudentAttendanceService.Models;
using StudentAttendanceService.Repository;

namespace StudentAttendanceService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentAttendanceController : ControllerBase
    {
        private readonly IStudentAttendanceRepository _repository;

        public StudentAttendanceController(IStudentAttendanceRepository repository)
        {
            _repository = repository;
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(_repository.GetAll());
        }

        [HttpGet("{id}")]
        public IActionResult GetById(int id)
        {
            var data = _repository.GetById(id);
            if (data == null) return NotFound();
            return Ok(data);
        }

        [HttpPost]
        public IActionResult Add(StudentAttendance attendance)
        {

```

```

        return Ok(_repository.Add(attendance));
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, StudentAttendance attendance)
    {
        var updated = _repository.Update(id, attendance);
        if (updated == null) return NotFound();
        return Ok(updated);
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        if (!_repository.Delete(id))
            return NotFound();

        return Ok("Deleted Successfully");
    }
}
}

```

POST /api/StudentAttendance	
Parameters	
No parameters	
Request body	
<pre>{ "studentID": 1, "semester": 6, "percentage_Attendance": 90 }</pre>	
200	Response body <pre>{ "studentID": 1, "semester": 6, "percentage_Attendance": 90 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 02 Mar 2026 04:25:57 GMT server: Kestrel</pre>

GET

/api/StudentAttendance

Parameters

No parameters

Execute

Code

Details

200

Response body

```
[
  {
    "studentID": 1,
    "semester": 6,
    "percentage_Attendance": 90
  },
  {
    "studentID": 2,
    "semester": 3,
    "percentage_Attendance": 80
  }
]
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 02 Mar 2026 04:27:33 GMT
server: Kestrel
```

PUT

/api/StudentAttendance/{id}

Parameters

Name	Description
id * required integer(\$int32) (path)	1

Request body

```
{
  "studentID": 1,
  "semester": 6,
  "percentage_Attendance": 95
}
```

200

Response body

```
{  "studentID": 1,  "semester": 6,  "percentage_Attendance": 95}
```

Response headers

```
content-type: application/json; charset=utf-8  date: Mon,02 Mar 2026 04:28:46 GMT  server: Kestrel
```

GET /api/StudentAttendance/{id}

Parameters

Name	Description
------	-------------

id * requiredinteger(\$int32)
(path)

Execute

200

Response body

```
{  "studentID": 1,  "semester": 6,  "percentage_Attendance": 95}
```

Response headers

```
content-type: application/json; charset=utf-8  date: Mon,02 Mar 2026 04:29:47 GMT  server: Kestrel
```

DELETE /api/StudentAttendance/{id}

Parameters

Name	Description
------	-------------

id * requiredinteger(\$int32)
(path)

Execute

200

Response body

Deleted Successfully

Response headers

content-type: text/plain; charset=utf-8
date: Mon, 02 Mar 2026 04:30:31 GMT
server: Kestrel

Extension : Use Ocelot gateway to perform route incoming request to particular Microservice. Read the corresponding lecture slides for developing understanding about Ocelot.

Ocelot.json:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/StudentAdmission",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5233
        }
      ],
      "UpstreamPathTemplate": "/admission",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
    },
    {
      "DownstreamPathTemplate": "/api/StudentAttendance",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5112
        }
      ],
      "UpstreamPathTemplate": "/attendance",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5015"
  }
}
```

Program..cs:

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Configuration
    .AddJsonFile("ocelot.json", optional: false, reloadOnChange: true);

builder.Services.AddOcelot();

var app = builder.Build();

await app.UseOcelot();

app.Run();
```

http://localhost:5015/admission

```
[
  {
    "studentID": 1,
    "name": "Rahul",
    "admissionDate": "2024-05-01T00:00:00",
    "admissionBranch": "CSE"
  },
  {
    "studentID": 2,
    "name": "Prashant",
    "admissionDate": "2025-07-01T00:00:00",
    "admissionBranch": "IT"
  }
]
```

http://localhost:5015/attendance

```
[
  {
    "studentID": 1,
    "semester": 6,
    "percentage_Attendance": 90
  },
  {
    "studentID": 2,
    "semester": 5,
    "percentage_Attendance": 76
  }
]
```